Лабораторная работа №23

Реализация функциональности приложения для ведения заметок

Основные правила

1. Не бойтесь **задавать любые** вопросы
2. Если хотите сделать что-то свое, то **предлагайте** - будем думать
3. Советую читать текст, тут написано много **полезного**

О чем лабораторная?

На прошлом занятии мы создали интерфейс приложения для ведения заметок. Сегодня добавим функциональность: создание и редактирование

Часть 1. Открываем выбранную заметку

Теперь сделаем так, чтобы при нажатии на карточку открывался экран NoteActivity.

Для этого адаптер должен сообщить MainActivity, на какую заметку нажал пользователь. Активность уже сможет открыть нужный экран.

Шаг 1. Добавляем обработчик нажатия в NoteAdapter

В `NoteAdapter` добавьте интерфейс и поле для обработчика:

```
public interface OnNoteClickListener {
    void onNoteClick(Note note, int position);
}

private OnNoteClickListener clickListener;
```

Теперь измените конструктор адаптера, чтобы он принимал не только список, но и **обработчик**:

```
public NoteAdapter(List<Note> noteList, OnNoteClickListener clickListener) {
    this.noteList = noteList;
    this.clickListener = clickListener;
}
```

В методе `onBindViewHolder()` после заполнения карточки добавьте обработку нажатия:

```
holder.itemView.setOnClickListener(v -> {
    clickListener.onNoteClick(currentNote, position);
});
```

Здесь `currentNote` - заметка, которую сейчас отображает карточка, а `position` - её позиция в списке.

Шаг 2. Подключаем обработчик в MainActivity

Добавьте в `MainActivity` поле для запоминания выбранной позиции:

```
int editedPosition = -1;
```

Также добавим в `MainActivity` метод для открытия экрана заметки:

```
private void openNoteActivity(Note note, int position) {
    // запоминаем карточку на которую нажали
    editedPosition = position;

    // создаем НАМЕРЕНИЕ с NoteActivity внутри
    Intent intent = new Intent(this, NoteActivity.class);

    // кладем внутрь данные по ключам
    intent.putExtra("NOTE_ID", note.getId());
    intent.putExtra("NOTE_TITLE", note.getTitle());
    intent.putExtra("NOTE_CONTENT", note.getContent());

    // запускаем наше НАМЕРЕНИЕ с кодом 101 (пригодится дальше)
    startActivityForResult(intent, 101);
}
```

Теперь, там, где вы создаёте адаптер, передайте ему обработчик нажатия:

```
adapter = new NoteAdapter(noteList, (note, position) -> {
    openNoteActivity(note, position);
});
```

> Проверьте результат: при нажатии на карточку открывается `NoteActivity` - **однако мы пока не заполняем поля в ней какими-то данными**.

Часть 2. Заполняем поля открытой заметки

Теперь передадим полученные данные в поля `EditText`.

В `NoteActivity` найдите метод `onCreate()`. После `setContentView()` найдите по `id` элементы интерфейса:

```
EditText editTitle = findViewById(R.id.editTitle);
EditText editContent = findViewById(R.id.editContent);
```

Затем получите данные из `Intent`:

```
// получаем намерение
Intent intent = getIntent();

// достаем из него данные
String title = intent.getStringExtra("NOTE_TITLE");
String content = intent.getStringExtra("NOTE_CONTENT");
```

Самостоятельно допишите две строки, которые установят полученные значения в поля ввода с помощью `setText()`.

Подсказка:

```
editTitle.setText(...);
```

> Проверьте результат: при открытии заметки заголовок и содержимое **автоматически появляются в соответствующих полях**.

Часть 3. Сохраняем изменения

Пользователь может изменить текст заметки. Теперь нужно вернуть новые значения в `MainActivity`.

Шаг 1. Создаём метод сохранения

В `NoteActivity` добавьте метод:

```
private void saveAndReturn() {
    // получаем введенные название и содержимое
    String title = editTitle.getText().toString().trim();
    String content = editContent.getText().toString().trim();

    // создаем НАМЕРЕНИЕ в которое положим наши данные
    Intent resultIntent = new Intent();
    resultIntent.putExtra("UPDATED_TITLE", title);
    resultIntent.putExtra("UPDATED_CONTENT", content);

    // СООБЩАЕМ в прошлый экран что результат ОК и отдаем данные
    setResult(RESULT_OK, resultIntent);

    // закрываем экран
    finish();
}
```

Обратите внимание:

- `getText().toString()` получает текст из поля ввода
- `putExtra()` сохраняет данные для передачи обратно
- `setResult()` устанавливает результат работы экрана

- `finish()` закрывает экран и возвращает пользователя на предыдущий

Подключите этот метод к кнопке "Сохранить":

```
Button buttonSave = findViewById(R.id.btnSave);

buttonSave.setOnClickListener(v -> {
    saveAndReturn();
});
```

Шаг 2. Получаем результат в MainActivity

В `MainActivity` добавьте поле для адаптера, если оно ещё не объявлено:

```
NoteAdapter adapter;
```

Затем добавьте метод `onActivityResult()`:

```
@Override
protected void onActivityResult(int requestCode, int resultCode, Intent data) {
    super.onActivityResult(requestCode, resultCode, data);

    // проверяем:
    // 1. если ответ пришел по 101 коду (вводили его ранее): requestCode == 101
    // 2. если результат - OK: resultCode == RESULT_OK
    // 3. если положили какие-то данные: data != null
    if (requestCode == 101 && resultCode == RESULT_OK && data != null) {
        // достаем из ответа строки по ключам
        String title = data.getStringExtra("UPDATED_TITLE");
        String content = data.getStringExtra("UPDATED_CONTENT");

        // получаем заметку на которую нажали
        Note note = noteList.get(editedPosition);

        // обновляем ее
        note.setTitle(title);
        note.setContent(content);

        // говорим адаптеру перерисовать карточку по данной позиции
        adapter.notifyItemChanged(editedPosition);
    }
}
```

Здесь мы получаем новые значения, изменяем существующий объект `Note` и сообщаем адаптеру, что карточку нужно обновить.

> Проверьте результат:
>
> - Откройте любую заметку

> - Измените заголовок или содержимое
> - Нажмите "Сохранить"
> - Убедитесь, что в списке появились новые значения

Пока мы работаем с **уже существующими заметками**. Создание новых добавим следующим этапом.

Часть 4. Создаём новую заметку

На главном экране уже есть кнопка "+". Сделаем так, чтобы она открывала пустую форму.

В `MainActivity` найдите кнопку добавления и подключите обработчик:

```
Button buttonAdd = findViewById(R.id.btnAdd);

buttonAdd.setOnClickListener(v -> {
    openNoteActivity(new Note(0, "", ""), -1);
});
```

Мы передаём пустую заметку с позицией `-1`. Это условное значение, означающее, что заметка ещё не создана.

Теперь нужно изменить `onActivityResult()`, чтобы он отличал создание от редактирования.

Найдите в нём строку:

```
Note note = noteList.get(editedPosition);
```

Замените код обновления заметки следующим фрагментом:

```
// если выбранная карточка с позицией -1
if (editedPosition == -1) {
    // создаем НОВУЮ заметка

    // получаем новый id
    int newId = noteList.size() + 1;

    // создаем заметку и добавляем в список
    Note newNote = new Note(newId, title, content);
    noteList.add(newNote);

    // оповещаем что добавили последний элемент
    adapter.notifyItemInserted(noteList.size() - 1);
} else {
    // редактируем уже СУЩЕСТВУЮЩУЮ

    // получаем ее из списка
    Note note = noteList.get(editedPosition);

    // обновляем
```

```
    note.setTitle(title);
    note.setContent(content);

    adapter.notifyItemChanged(editedPosition);
}
```

Теперь программа работает по-разному в зависимости от действия пользователя.

> Проверьте результат:
>
> - Нажмите "+"
> - Введите заголовок и содержимое
> - Сохраните заметку
> - Убедитесь, что новая карточка появилась в списке

Часть 5. Удаляем заметку

Теперь самостоятельно реализуйте удаление заметки. На экране `NoteActivity` уже есть кнопка "Удалить", осталось сделать так, чтобы она действительно работала.

Вспомните, как **мы сохраняли заметку и возвращали результат** в `MainActivity`.

При удалении будем действовать похожим образом.

Шаг 1. Обрабатываем нажатие на кнопку

В `NoteActivity` найдите кнопку "Удалить" и добавьте обработчик нажатия.

При нажатии программа должна:

1. Сообщить `MainActivity`, что пользователь хочет удалить заметку
2. Закрыть текущий экран

Подсказки:

- Для передачи результата используйте `setResult()`
- В качестве кода результата можно использовать число `100`
- Чтобы закрыть экран, используйте `finish()`

Шаг 2. Обрабатываем удаление в MainActivity

Теперь нужно научить `MainActivity` реагировать на результат удаления. Откройте метод `onActivityResult()`, который мы создали в предыдущей части.

Добавьте проверку, которая определяет, что пользователь запросил удаление заметки.

Если получен код `100`, программа должна:

1. Проверить, что была открыта существующая заметка, а не форма создания новой
2. Удалить заметку из noteList по сохранённой позиции editedPosition
3. Сообщить адаптеру об удалении элемента с помощью notifyItemRemoved()

Подсказки:

Удалить элемент из списка можно так:

```
noteList.remove(позиция);
```

Сообщить адаптеру об удалении:

```
adapter.notifyItemRemoved(позиция);
```

Для проверки кода результата используйте условие:

```
resultCode == 100
```

> Проверьте результат
>
> 1. Запустите приложение и откройте существующую заметку
> 2. Нажмите "Удалить"
> 3. Убедитесь, что экран заметки закрылся, а карточка исчезла из списка
> 4. Создайте новую заметку и проверьте, что её случайно нельзя удалить до сохранения

Важно: при создании `editedPosition` равна -1. Поэтому нельзя удалять элемент списка, если позиция равна -1 - его попросту нет в списке

Итоговые требования

1. Приложение отображает список заметок с помощью `RecyclerView`
2. При нажатии на карточку открывается экран выбранной заметки
3. Поля ввода заполняются данными существующей заметки
4. Можно создавать новые заметки
5. Можно изменять заголовок и содержимое существующих заметок
6. Можно удалять заметки
7. Список обновляется после создания, изменения и удаления

Дополнительно

1. Сделайте так, чтобы при нажатии кнопки "Назад" **изменения не сохранялись**
2. **Запретите сохранение** заметки, если её заголовок пустой
3. **Добавьте подтверждение** перед удалением заметки (можно использовать `AlertDialog`)